David Silver 强化学习课程笔记

Lecture1: Introduction to Reinforcement Learning



SpiritsYouthHarmony

2026 年 3 月

1 “Where” is RL ?

强化学习本质上是一种决策科学（Science of Decision-making），但其实在计算机、工程学、神经科学、数学、经济学、心理学中都有它的身影，如图 1。

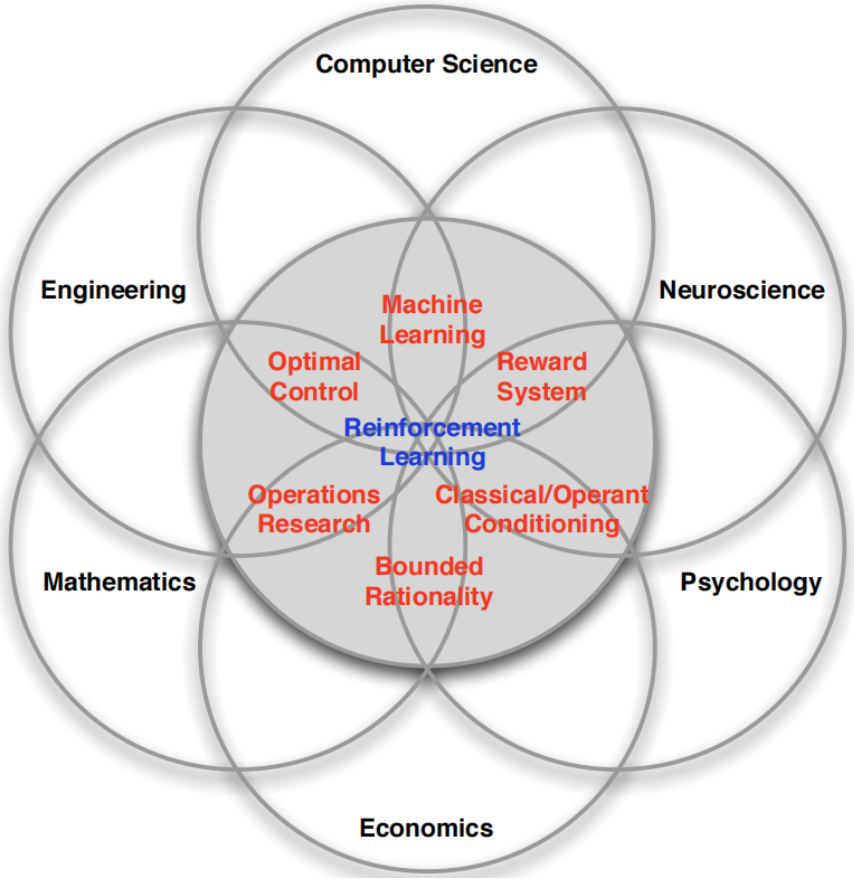


图 1: RL 在各个学科中对应的位置

- 计算机：RL 属于 ML 的范围，是一种行为主义的学习方法。
- 工程学：RL 和最优控制（Optimal Control）关注的问题一样，即如何选择一系列动作，以便最终获得最优结果。
- 数学：研究最优控制的方法，被称为运筹学（Operations Research）
- 神经科学：人脑的奖励系统现在被广泛研究，多巴胺的分泌能够刺激大脑，这和 RL 中奖励信号能够指导 agent 做出动作是一致的。
- 心理学：多年来对于条件反射的研究一直是一个焦点问题。给受试动物食物，让它们形成某种条件反射，在 RL 中就是给 agent 奖励让它去选择某种动作。
- 经济学：博弈论（Game Theory）中的三要素 player、strategy、utility 和 RL 的 agent、action、reward 本质上一致。有限理性（Bounded Rationality）也是一个重点问题。

2 RL VS other ML

RL 和其他的机器学习方法有什么区别？

1. RL 解决的是序贯决策的问题，时间非常重要，因为决策过程基本都是 MDP，后来的状态、动作、奖励都会受到之前时刻的影响，也就是说数据不是 i.i.d 的。
2. RL 没有监督信号（没有 ground-truth label），只有 reward（奖励）信号。并且，奖励信号很可能是延迟性的，比如走迷宫，只有在走出的时候会得到一个 reward，在迷宫中探索的时候没有任何信号。
3. 智能体的动作影响环境，也就是说整个过程是 dynamic 的。

3 The RL Problem

首先，我们举一些例子来说说强化学习有哪些可能的应用，直观地感受一下。

- 直升机运动控制：直升机模型通过强化学习学习执行特定的特技飞行，如倒飞、旋转等。
- 玩 Atari 游戏：通过看一帧帧的游戏图像，决定下一时刻如何操控游戏手柄，以达到任务目标。
- 管理投资：agent 实时决定投资哪些产品，最大化利益。

OK，下面，我们用一些概念来更清楚地定义 RL 问题：

Reward（奖励）、State（状态）、Agent & Environment（智能体与环境的交互）

1. **Reward（奖励）**

Definition (Reward Hypothesis)

All goals can be described by the maximization of expected cumulative reward.

首先，在强化学习里，有这样一个前提假设，所有的目标都可以通过最大化 expected cumulative reward（期望累计奖励）来达到。

这里当我们谈到奖励（以 R_t 表示）时，R_t 是一个**标量**反馈信号，反映出 agent 在 t 时刻的行为是不是一个好的动作。

举例来说，在直升机运动控制案例里，如果直升机飞出了我们期望的轨迹，就可以给它一个足够大的正值奖励；如果飞机发生碰撞或者坠落，就给它一个负值奖励。在管理投资问题里，奖励信号非常清晰，比如美元或英镑。在这种情况下，目标就是简单地最大化总奖励。又比如在玩 Atari 游戏的例子中，在每个时间步对分数的每次变化给予正或负奖励（比如如果你在 t 时刻走那一步之后得到 10 分，那就在该时间步给予 +10 的奖励。

其实，这些问题看起来是完全不同的类型，但是在 RL 中，统一把它们放在一个框架下，用相同的形式化处理所有这些不同类型的问题，从而用相同的思想来解决它们。这里，第一步就是要明确在每个时间步获得的奖励信号。

2. **Sequential Decision Making（序贯决策）**

上一小节讲 RL 和其他 ML 不同的时候，第一条就说明了，RL 研究的是序贯决策问题。这种问题的目标是让 agent 选择 action（动作），以最大化 total future reward（未来的总奖励）。特别要注意的

一点是，reward 具有延迟性，这意味着我们必须提前规划、提前思考，因为奖励可能不会立即到来，而可能在未来的某个时间步。有时这甚至意味着你可能需要放弃眼前的一些奖励，以便在未来获得更多奖励。所以做强化学习时不能短视，必须着眼长远。

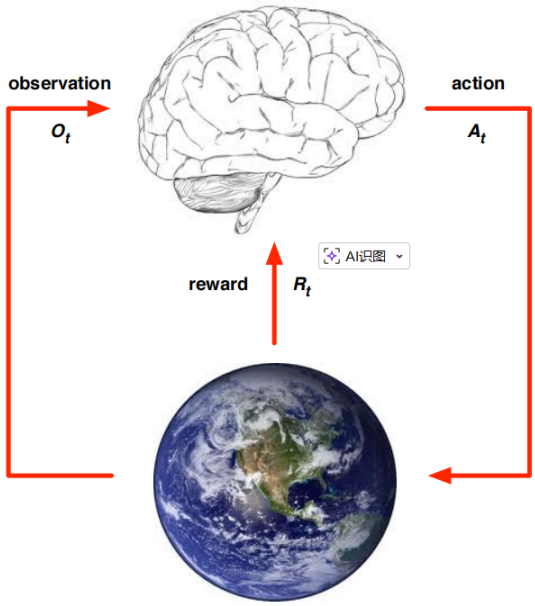


图 2: Agent and Environment

3. **Agent and Environment（智能体与环境）**

   如图 2，这应该是最能代表 RL 的一张图了。在每个时间步 t，agent 会执行动作 A_t，接收观测信号 O_t 和奖励信号 R_t；与此同时，environment 会收到 agent 的动作信号 A_t 并向 agent 反馈观测信号 O_{t+1} 和奖励信号 R_{t+1} 智能体与环境之间的这种交互不断循环进行。因此，强化学习中 trial-and-error loop 基本上是一个由观察、奖励和行动组成的时间序列。这个时间序列定义了智能体的 experience（经验），而这个经验就是我们用于强化学习的数据。

4. **History and State（历史和状态）**

   历史被定义为观测、动作和奖励的序列：

$$H_t = O_1, R_1, A_1, \ldots, A_{t-1}, O_t, R_t \tag{1}$$

   也就是说，历史是截止 t 时刻 agent 所看到的一切。每个时间步它采取行动，看到 observations ，获得奖励，一直到当前时间步。历史就是截止 t 时刻 agent 看到的一切的序列。

   然而，历史包含的信息可能会特别大，所以，我们引入 state（状态）这个概念。状态就像是一个信息摘要，用于决定接下来会发生什么。state 让我们能够用一个量来替代历史，从而做出决策。

$$S_t = f(H_t) \tag{2}$$

   状态这个概念非常关键，但很容易被误解。一般来说，如 2所示，状态是历史的一个函数，可以是任意一个映射。比如说，在玩 atari 游戏时，状态被定义为当前时刻 t 以及 t-3、t-2、t-1 时刻共四帧对应的图像信息。

这里需要明确 Environment State（环境状态）、Agent State（智能体状态）、Information State（信息状态 / 马尔可夫状态）三个概念的区别。

Environment State (S_t^e)	环境状态是环境私有的信息，对于 agent 来说通常不可见，即使可见，也可能包含很多冗余的信息。比如，一个机器人在房间里学走路，它并不知道房间里那杯透明的液体到底是什么（也许是水、也许是过氧化氢溶液），它只能通过摄像头看到一个小小的画面而已，对于这整个环境完全不了解。
Agent State (S_t^a)	智能体状态是我们在 RL 里会用到的概念，它实际上就是在算法里存储的一堆数字，这些数字捕获了智能体截止 t 时刻所经历的一切，更重要的是，智能体会根据这些数字来选择下一个动作。
Information State	信息状态是一个数学概念，又称马尔可夫状态。这个状态意味着在构建一个状态表示时，需要包含来自历史的所有有用信息。如 3 所示，“The future is independent of the past given the present”，你只需要存储信息状态，因为这个状态包含足够的信息以表征关于未来的一切——所有未来的观测、行动、奖励。

表 1: 三种状态的对比

Definition (Markov)

A State S_t is Markov if and only if

$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, \ldots, S_t] \tag{3}$$

5. **Two Special Concepts（两个特殊的概念）**

Fully Observable Environments：完全可观测环境，这代表智能体能看到环境中发生的一切，这是我们在课程剩余部分大部分时间里要处理的情况。这是一个很好的情况，如 4，此时环境状态、智能体状态和马尔可夫状态是等价的。当我们使用这种类型的表示时，我们可以得到强化学习的主要形式化方法，也就是马尔可夫决策过程（Markov Decision Process, MDP）。

$$O_t = S_t^a = S_t^e \tag{4}$$

Partially Observable Environments：部分可观测环境，这代表智能体不能完全知晓环境的所有信息，例如一个只带摄像头的机器人，它不知道自己的确切位置，因为它没有 GPS。在这种环境里，agent state $\neq$ environment state，其形式化方法为部分观测的马尔可夫决策过程（Partially Observable MDP, POMDP）。

对于这类情形，agent 必须要建立起自己的 S_t^a，一般有以下三种方法：

- 直接把完整的历史信息作为 agent state：$S_t^a = H_t$
- 根据对 environment 的信念来建立：$S_t^a = (\mathbb{P}[S_t^e = s^1], \ldots, \mathbb{P}[S_t^e = s^n])$
- 神经网络拟合

4 Major Components of an RL Agent

一个 RL agent 包括下列三个要点中的一个或多个：

Policy、Value Function、Model

1. **Policy（策略）**

   “A policy is the agent’ s behaviour”，也就是说，策略本质上是对某个 agent 从状态到动作的映射。策略可以是 deterministic（确定性的）也可以是 stochastic（随机的）。确定性策略意味着给定状态，agent 选择某个动作是确定的，用 $a = \pi(s)$ 表示；随机性策略表示给定状态，agent 以不同概率选择不同的动作，用 $\pi(a \mid s) = \mathbb{P}[A_t = a \mid S_t = s]$ 表示。

2. **Value Function（价值函数）**

   “Value function is a prediction of future reward”，不管是状态 s 还是动作 a，都会有一个对应的价值来表示其好坏，形式上，当策略给定，对于一个状态 s 来说，它的价值如5：

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s] \tag{5}$$

3. **Model（模型）**

   “A model predicts what the environment will do next”，模型的作用是建立起对环境的一种猜测，一种对概率分布的认识，从而指导 agent 选择动作。

   模型包括 **Transition Model（转移模型）和 Reward Model（奖励模型）**。如 6，转移模型根据当前状态和采取的动作预测下一个状态将是什么。比如说，当前直升机在 5m 高空，动作是向下飞，那么转移模型会预测下一个状态就是 4m 的位置。

   如 7，奖励模型用于预测在状态 s 下采取动作 a 可能获得多少奖励。

$$\mathcal{P}^a_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a] \tag{6}$$

$$\mathcal{R}^a_s = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] \tag{7}$$

下面来看一个例子：迷宫。

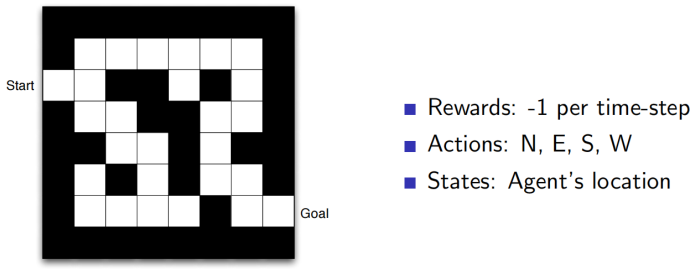


图 3: Maze Example

如3，目标是从 Start（起点）走到 Goal（终点），每走一步给一个-1 的奖励值，可选的动作包括上下左右（N，E，S，W），状态就是 agent 所处的位置坐标。

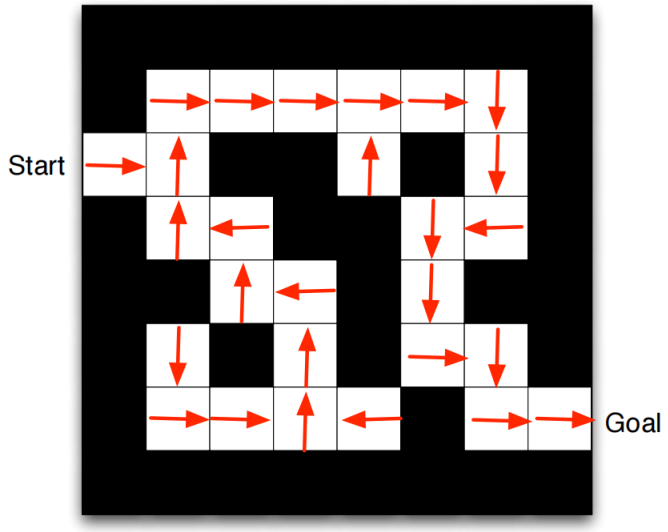


图 4: Policy Example

对于 Policy 来说，举个 deterministic policy 的例子，如4，每个格子（也就是每个状态）都有一个方向选择（红色的箭头，相当于动作），这构成了一种映射 $a = \pi(s)$ 。

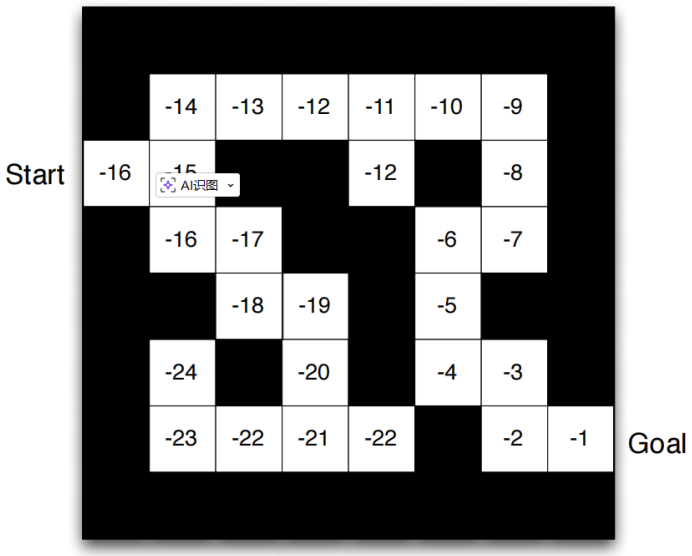


图 5: Value Function Example

对于 Value Function 来说，如5，每个位置（state）有一个数字，这表示 state value，例如，离 Goal 最近的一格值为-1，就是说再走一步就能到达目标了，离越远 value 越小，这引导 agent 朝着接近 Goal 的方向移动。

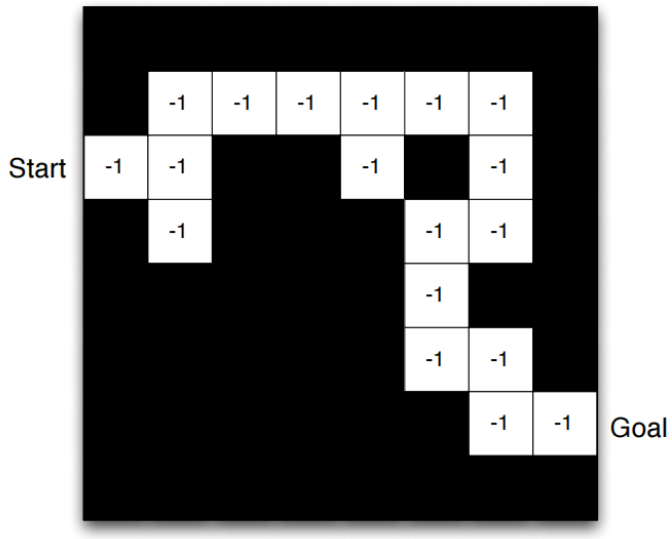


图 6: Model Example

对 Model 来说，模型可能如6，agent 会根据自己的观测、动作、奖励等信息构建出对迷宫的一个“想象”，包括转移模型（表示 agent 对于自己在某个格子，行动以后下一时刻会处于哪个格子的一种认知）和奖励模型（表示 agent 对于自己在某个格子，选取某个动作会带来多好的结果的认知）。注意，模型很可能是不完美的，不等于真实的现实环境。

5 Problems within Reinforcement Learning

1. **Learning and Planning（学习与规划）**

   对于 RL 而言，环境一开始是未知的，agent 通过不断与环境交互，根据获得的奖励和观测信号来不断修正自己的策略，以达到最大化期望奖励的目标。所以，RL 是一种 trial-and-error 的过程。比如让计算机学玩 atari 游戏。

   对于规划问题，环境的模型是非常清楚的，也就是说第一性原理是明确的。在这类问题里，agent 要做的是不断根据模型做计算，从而不断选择好的策略。比如利用 minimax 算法下棋，只要根据规则和模型一步步选择最好的位置落子即可。

2. **Exploration vs Exploitation（探索与利用）**

   RL 是一种 trial-and-error 的过程，我们选择了某个动作可能带来很大的奖励，那么是否意味着我们应该倾向于选择这个动作？或者说，如果我们选择某个动作带来了很大的惩罚，那么是否意味着我们应该倾向于拒绝这个动作？这个问题是一个探索和利用的平衡问题。

   Exploitation（利用）意味着利用已知信息，以最大化奖励。比如，去你最爱的那家店点最爱的菜。虽然保证好吃，但你永远不知道隔壁街是不是开了家更好的。

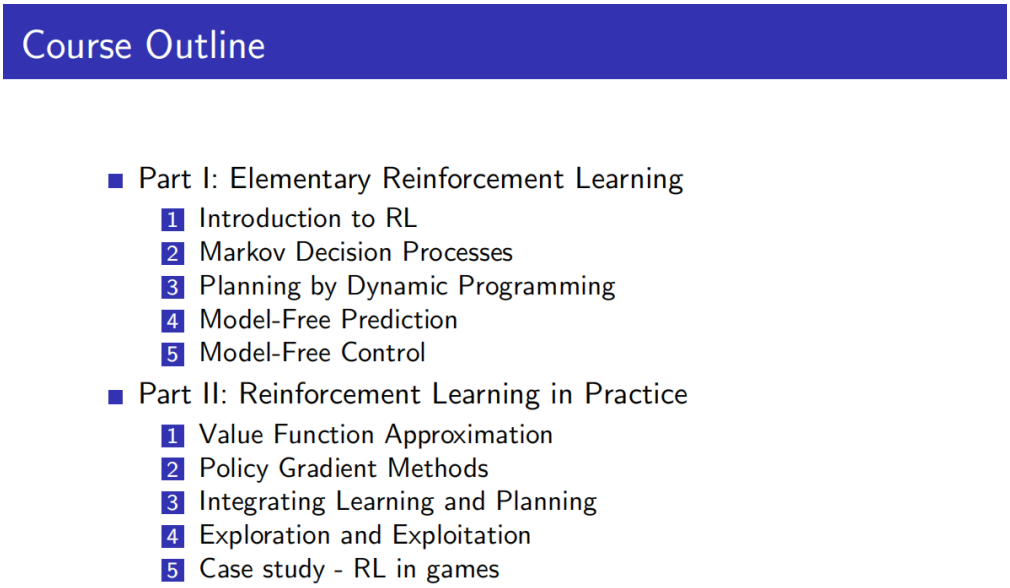
   Exploration（探索）意味着选择放弃一些已知的奖励，以便建立出更多对于环境的认知。比如，冒险去尝试一家零评论的新店。可能发现人间美味，也可能极其难吃。

   RL 的独特之处就在于它必须在交互中平衡这两者。

3. **Prediction vs Control（预测与控制）**

   这两个概念对应后续的 policy evaluation 和 policy improvement，预测是说对给定一个固定的策略 π，我们要计算出在这个策略下，每个状态的价值函数 $v_\pi(s)$ 是多少；控制的目标是寻找最优策略 π^*，使得累积奖励最大化。事实上，Prediction 是 Control 的基础。我们必须先准确评估当前策略的价值，才能在这个基础上推导出更优的策略。

OK，这就是第一节课的全部内容了，比较全面地把“RL 研究什么问题，其中有哪些要素，RL 问题关注哪些点”这些内容给进行了介绍，后续课程如下图，会进一步详细介绍 RL 的基础知识和实际应用。



参考文献

[1] David Silver. *Lectures on Reinforcement Learning*. URL: `https://www.davidsilver.uk/teaching/`, 2015.